```
# Import required libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# Load the Iris dataset from a public URL
url =
"https://raw.githubusercontent.com/uiuc-cse/data-fa14/gh-pages/data/
iris.csv"
data = pd.read_csv(url)

# Separate features (X) and target (y)
X = data.drop('species', axis=1)
y = data['species']

# Split data into training and testing sets (80/20)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# --- Feature Scaling (for KNN) ---
# KNN is sensitive to feature scales, so we scale the data.
# Decision Trees do not require feature scaling.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# --- 1. Decision Tree Model ---
# Initialize the model (using 'entropy' criterion as shown in the doc)
dt_model = DecisionTreeClassifier(criterion='entropy',
random_state=42)

# Train the model on the *unscaled* training data
dt_model.fit(X_train, y_train)

# Make predictions on the *unscaled* test data
y_pred_dt = dt_model.predict(X_test)

# --- 2. K-Nearest Neighbors (KNN) Model ---
# Initialize the model (using 5 neighbors)
knn_model = KNeighborsClassifier(n_neighbors=5)

# Train the model on the *scaled* training data
knn_model.fit(X_train_scaled, y_train)

# Make predictions on the *scaled* test data
```

```
y_pred_knn = knn_model.predict(X_test_scaled)

# --- Evaluate Decision Tree ---
print("\nDecision Tree Results:")
print("Accuracy:", round(accuracy_score(y_test, y_pred_dt)*100, 2),
"%")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_dt))
print("Classification Report:\n", classification_report(y_test,
y_pred_dt))

# --- Evaluate KNN ---
print("\nKNN Results:")
print("Accuracy:", round(accuracy_score(y_test, y_pred_knn)*100, 2),
"%")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_knn))
print("Classification Report:\n", classification_report(y_test,
y_pred_knn))
```

```
Decision Tree Results:
Accuracy: 100.0 %
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30


KNN Results:
Accuracy: 100.0 %
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11
```

```
    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```